Kafe: Can OS Kernels Forward Packets Fast Enough for Software Routers?

Cheol-Ho Hong, Kyungwoon Lee, Jaehyun Hwang, Hyunchan Park, and Chuck Yoo, *Member, IEEE, ACM*

***Abstract*— It is widely believed that software routers based on commodity operating systems cannot deliver high-speed packet processing, and a number of alternative approaches (including user-space network stacks) have been proposed. This paper revisits the inefficiency of kernel-level packet processing inside modern OS-based software routers and explores whether a redesign of kernel network stacks can improve the incompetence. We present a case contrary to the belief through a redesign: Kafe—a kernel-based advanced forwarding engine that can process packets as fast as user-space network stacks. The Kafe neither adds any new API nor depends on proprietary hardware features, but the Kafe outperforms Linux by seven times and RouteBricks by three times. The current implementation of the Kafe can forward 64-byte IPv4 packets at 28.2 Gbps using eight cores running at 2.6 GHz. Our evaluation results show that the Kafe achieves similar packet forwarding performance to Intel DPDK while consuming much less CPU and memory resources.**

***Index Terms*— Software router, OS network stack, optimization.**

I. Introduction

FOR a decade, software routers have gained significant attention because of their strong programmability, extensibility, and high cost effectiveness. As they are based on commodity hardware and modern operating systems (OSs), they provide rich functionalities extensible by familiar programming environments. Owing to this advantage, new features can be added to software routers without deploying additional hardware. In addition, the cost of PCs and network interface cards (NICs) is decreasing rapidly because of the progress and competition in the hardware market. Therefore, nowadays software routers have become an appealing alternative to inflexible and expensive proprietary hardware routers [1]–[11].

While software routers are programmable, extensible, and highly cost effective, high-speed packet forwarding has always been an issue because of bottlenecks in modern OSs [1]–[4], [7], [10]–[12]. For example, Linux could forward 64-byte IPv4 packets at a mere rate of approximately 0.7 Mpps (million packets per second) per core in our experiments (Section IV-B). This is far from saturating a single 10 Gbps NIC as it is fully saturated at 14.88 Mpps. This poor performance is because Linux was originally designed for general purpose and feature rich networking, not high-speed packet processing.

Consequently, quite a few research systems bypass the OS kernel and implement their networking stacks in user space [7], [13]–[15]. However, user-space networking suffers from lack of protection so that application bugs can corrupt the networking stack itself. To remedy this problem, a hybrid approach called IX [16] has been proposed recently in which IX puts the data plane in protected mode and applications in user mode so that they work together to outperform state-of-the-art user-space network stacks. Although it has many benefits, IX introduces a new set of APIs that applications are required to use in order to take advantage of its benefits.

Although user-space network stacks and the hybrid approach achieve outstanding performance, leveraging modern OSs for packet forwarding has several significant advantages. Modern OSs provide reliable resource management, full-fledged kernel network stacks, and compatibility with existing IP routing applications for real-world deployment. Motivated by the advance in modern network interface cards and multi-core technologies, this paper revisits the issue of kernel network stacks inside modern OS-based software routers. We investigate the source of inefficiency in packet forwarding and explore whether a redesign of kernel network stacks can improve the incompetence.

In this paper, we present *Kafe*, a kernel-based advanced forwarding engine for software routers that can process packets as fast as user-space network stacks. Kafe overcomes the traditional perception that the network architecture of modern OSs is complicated to revise and difficult to fully utilize the line speed. We introduce two new mechanisms: the skb Stack and the Flow Repository. The skb Stack is a cache-optimized socket buffer (skb) allocator that recycles pre-allocated buffers with hardware cache efficiency while preserving the semantics of the skb structure. The Flow Repository stores kernel objects that can be cached across multiple packets that belong to the

Manuscript received January 16, 2017; revised January 19, 2018 and July 23, 2018; accepted October 3, 2018; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor Y. Chen. Date of publication November 21, 2018; date of current version December 14, 2018. This work was supported by the Institute for Information & Communications Technology Promotion (IITP) Grant 2015-0-00280 funded by the Korea government (MSIT), in part by the (SW Starlab) Next generation cloud infra-software toward the guarantee of performance and security SLA; in part by the Research of Network Virtualization Platform and Service for SDN 2.0 Realization under Grant 2015-0-00288; and in part by the National Research Foundation of Korea (NRF) Grant 2010-0029180 funded by the Korea government (MSIT). *(Corresponding author: Chuck Yoo.)*

C.-H. Hong is with the School of Electrical and Electronics Engineering, Chung-Ang University, Seoul 06974, South Korea (e-mail: cheolhohong@cau.ac.kr).

K. Lee and C. Yoo are with the Department of Computer Science and Engineering, Korea University, Seoul 02841, South Korea (e-mail: kwlee@os.korea.ac.kr; chuckyoo@os.korea.ac.kr).

J. Hwang is with Samsung Electronics, Suwon 16677, South Korea (e-mail: jhyun.hwang@samsung.com).

H. Park is with the Department of Computer Science and Engineering, Chonbuk National University, Jeonju 54896, South Korea (e-mail: hyunchan.park@chonbuk.ac.kr).

Digital Object Identifier 10.1109/TNET.2018.2879752

TABLE I
MAJOR BOTTLENECKS WITH EACH CPU CYCLES CONSUMED IN CONSECUTIVE 64-BYTE IPV4 PACKET FORWARDING IN LINUX

Category	Symbols	Cycles	Our solution (Kafe)
Routing lookup	fib_table_lookup, check_leaf.isra.8, ip_route_input_noref, etc.	20.25%	Flow Repository (§III.C)
RX and TX descriptor processing	ixgbe_clean_rx_irq, ixgbe_xmit_frame_ring, __dev_queue_xmit, etc.	16.80%	skb Stack (§III.B)
skb allocation and de-allocaton	kmem_cache_alloc, build_skb, __slab_free, __slab_alloc , etc.	13.06%	skb Stack (§III.B)
Firewall processing	nf_iterate, ipt_do_table, selinux_ip_forward, selinux_ip_postroute, etc.	9.48%	Flow Repository (§III.C)
Protocol retrieving	eth_type_trans	4.44%	Flow Repository (§III.C)
Neighbor lookup	ip_finish_output2	3.26%	Flow Repository (§III.C)
Other L3 operations	dev_gro_receive, ip_rcv, ip_forward, udp_v4_early_demux, inet_gro_receive, etc.	11.17%	-
Other operations	_raw_spin_lock, memcpy, etc.	21.54%	-
Total		100%	

same flow. These two mechanisms contribute to a redesign of kernel network stacks by replacing the per-packet processing design of modern OSs with a recycle-based design that exploits pre-allocated and cached objects. The new architecture allows each packet to pass through the Link and Network Layers without per-packet inspection and object allocation.

We develop these two new mechanisms through a thorough analysis of packet forwarding in kernel network stacks. Our analysis is different from previous studies in that our analysis covers the full spectrum of packet forwarding as in Table I whereas previous studies only focus on the reception and transmission parts of device drivers [4], [7]. Based on the two mechanisms, Kafe achieves a high packet forwarding rate as fast as existing user-space network stacks. Currently, Kafe runs at 28.2 Gbps for IPv4 with 64-byte packets using eight cores running at 2.6 GHz, which is seven times faster than Linux and three times than RouteBricks [4]. Kafe also shows similar performance to Intel DPDK [15], a typical data plane for user-space solutions.

Another advantage of Kafe is that it consumes much less CPU and memory resources because Kafe does not use continuous polling or huge packet buffers. Therefore, in comparison with existing systems that use polling or huge buffers, the resource consumption of Kafe is very small: 1/5 of RouteBricks and 1/100 of Intel DPDK in terms of memory use. Additionally, Kafe neither introduces any new APIs nor depends on proprietary hardware features. Kafe is therefore fully compatible with any application and allows easy integration with new development such as Open vSwitch (OVS) [17].

The remainder of this paper is structured as follows: In Section II, we explain the background of the packet forwarding process and our motivation. In Section III, we elaborate on the detailed designs of Kafe. In Section IV, we show the performance evaluation results. Section V presents discussion points. In Section VI, we describe the related work. Finally, we present our conclusions in Section VII.

II. BACKGROUND AND MOTIVATION

In this section, we will explain how packet processing is performed in kernel space of software routers. Our explanation is based on Linux and the Intel *ixgbe* NIC driver [18], [19], a representative driver in the literature on software router research [4], [7], [8]. The whole process is summarized in Fig. 1. Afterward, we will explain our motivation.

A. Packet Forwarding in Kernel Space

A NIC maintains a pair of RX (reception) and TX (transmission) circular queues that contain RX and TX descriptors, respectively. Each descriptor includes the packet buffer address, the packet length, and some status fields. As each descriptor is accessible by both a core and a NIC, when one of them modifies the descriptor, the other can identify the change. Before the reception, the host prepares several RX descriptors in advance along with packet buffers in the system memory. On packet reception, the NIC fetches the next available RX descriptor and copies the contents of the incoming packet into the system memory advised by the packet buffer address field of the fetched RX descriptor. Then, the NIC records the packet length and some status fields in the RX descriptor according to the incoming packet. Recent multi-queue NICs allow a host to configure a pair of RX and TX queues per core for scalability. In this configuration, a NIC can utilize Receive Side Scaling (RSS), which distributes packets between different RX queues based on their flow information [20], [21].

In the reception part of the driver (the two left Link Layer boxes in Fig. 1), NAPI (a new API), which combines interrupt and polling mechanisms, is generally used to deal with high traffic loads. After the first interrupt, the poll function of the driver (*clean_rx_irq* in *ixgbe*) disables the subsequent interrupts and performs the task of fetching RX descriptors from the RX queue. After fetching a descriptor, the poll function fills an allocated *sk_buff* (skb) object with the contents of the RX descriptor and the packet. Next, the poll function calls a NIC-dependent receive function (*ixgbe_rx_skb* in *ixgbe*) and *netif_receive_skb*. They process TCP Segmentation Offload (TSO) [22] or Generic Segmentation Offload (GSO) [23] and initialize the Network Layer boundary fields of the skb. Later, an appropriate L3 layer protocol handler (*ip_rcv* in the case of IPv4), which is retrieved from the header of the packet, is executed. The poll function iteratively fetches RX descriptors within its budget, which restricts the maximum number of packets it can process each time, for fairness between other network devices. Before termination, it enables interrupt notifications for subsequent pending frames.

In Fig. 1, the L3 layer boxes perform the Network Layer tasks, which include IP header checksum validation, destination address interpretation, time-to-live (TTL) decrement, and other miscellaneous operations. IP header checksum validation

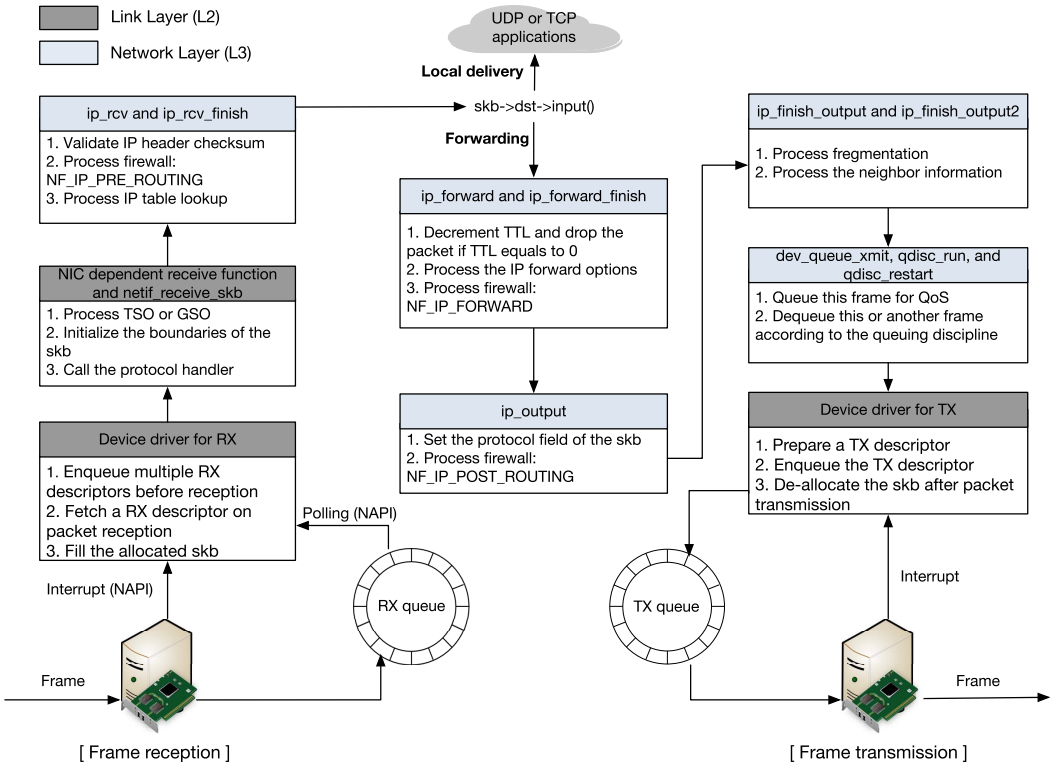

Fig. 1. Packet forwarding process in Linux. The L2 boxes on the left represent the reception part of the device driver. The L3 boxes denote the Network Layer tasks. The L2 box on the right represents the transmission part of the device driver.

verifies that the checksum value of the packet has not been corrupted before reception. This task is performed by *ip_rcv*. Destination address interpretation explores the routing table to determine which of the output interfaces leads to the destination address of the packet. *ip_rcv_finish* performs this task. TTL decrement decreases the TTL field in the header by one. If the field reaches zero, the packet is discarded [24]. This work is executed by *ip_forward*. Other operations include firewall, IP fragmentation, IP options, error check, and quality of service (QoS) processing, which are not standard for all packets, but ready for special cases. They are performed across the L3 layer functions in the Network Layer boxes.

In the transmission part of the driver (the right L2 box in Fig. 1), the transmission function fetches the next available TX descriptor and assigns the packet buffer address and the packet length fields to the descriptor, which are required for copying the packet contents to the NIC. *ixgbe_xmit_frame* in *ixgbe* performs this task. This function then inserts the TX descriptor into its corresponding TX queue. The outgoing packet is copied to the NIC asynchronously by the DMA engine. After this packet leaves the host, the NIC generates an interrupt for letting the driver start the clean-up process. The interrupt handler of the driver (*clean_tx_irq* in *ixgbe*) then de-allocates the skb object and its packet buffer.

B. Overheads in Kernel-Level Packet Forwarding

We analyze the overheads incurred during packet forwarding based on recent Linux and the Intel *ixgbe* driver. PacketShader [7] and RouteBricks [4] analyzed the overheads focusing on the RX and TX driver parts respectively, but we take a holistic approach to analyze the entire packet forwarding path. For this analysis, we use an Intel Xeon E5-2650 v2 octa-core platform running at 2.6 GHz with 20 MB of L3 cache and 64 GB of main memory. For network cards, we use two dual-port Intel 82599EB 10GbE NICs. The system is hosted by Linux with kernel version 3.16.4. The version of the Intel *ixgbe* driver is 3.22.3. We use another server that runs Pktgen from WindRiver [25] to generate and receive network traffic with 64-byte IPv4 packets. Our analysis measured the CPU cycles for the entire packet forwarding path, and Table I summarizes the results. Table I shows the categories in the forwarding path, key functions in each category, percentages of CPU cycles, and Kafe mechanisms that can improve each category.

We identified that routing lookup consumes 20.25% of the total CPU usage. In Linux, the routing information is stored in the Forwarding Information Base (FIB), which is implemented using a trie data structure. The FIB lookup algorithm that explores the trie structure in *fib_table_lookup* is quite complex and comprehensive, which incurs significant costs. Moreover, the routing cache subsystem that can help this complexity has been removed from Linux 3.6, as it was inefficient.

Both the RX and TX descriptor processing parts take up 16.80% of the total CPU cycles. Each part includes fetching a descriptor from the RX or TX queue, changing the buffer address to its matching DMA address, and inserting the descriptor to the RX or TX queue. In particular, after careful profiling, we have identified that per-packet DMA address translation has considerable CPU cycles.

The skb allocation and de-allocation take up 13.06% of the total CPU usage. The (de-)allocation functions are dependent

on the SLAB memory allocator [26], which is not optimized for frequent per-packet memory allocation [7]. As the allocator has to deal with several millions of buffer allocations and de-allocations per second for consecutive 64-byte packet forwarding, it becomes one of the major bottlenecks.

We see that firewall processing takes up 9.48% of the total CPU cycles. Firewall processing constructs a set of hooks inside the Linux kernel. Then, each registered hook function is called for every packet that traverses the network stack. As at least three hook functions are registered and executed per packet, as shown in Fig. 1, firewall processing incurs significant expenses.

Protocol retrieving consumes 4.44% of the total CPU cycles, and is performed by the *eth_type_trans* function that determines the L3 layer protocol of a packet. As this function accesses the Link Layer header to identify the protocol for every packet, it steadily incurs compulsory cache misses.

Neighbor lookup takes up 3.26% of the total CPU cycles. The Linux neighboring subsystem maintains the ARP cache implemented as a hash table to boost translation from the L3 address to its L2 address. To use the cache, the flow information should be transformed into a hash value by the ARP hash function. The lookup for the hash table also needs non-negligible computation per packet to iterate hash chains.

The remaining L3 layer operations excluding the aforementioned bottlenecks, take up 11.17% of the total CPU cycles. These operations include processing Generic Receive Offload (GRO) [27], IP header checksum validation, TTL decrement, IP options processing, and UDP assistance.

C. User-Level Networking Approaches

To overcome the overheads in kernel-level packet processing, several research systems [7], [28], [29] bypass the kernel and implement their networking stacks in user space based on general networking frameworks such as netmap [8] and Intel DPDK [15]. Nevertheless, following limitations of user-level approaches motivate us to take a kernel-level approach:

First, most user-level networking stacks are not competitive in terms of implemented features [30]. For example, DPDK is a typical networking framework for user-level stacks and provides a set of data plane libraries and drivers for fast packet processing. However, DPDK does not offer networking stacks and functions such as L3 forwarding and firewall processing [15]; it only provides simple example applications. Therefore, user-level solutions based on DPDK need to build a new networking stack from scratch or the limited examples. Implementing full-featured networking functions for user-level solutions is definitely a complicated task, given that network functions in a modern OS have been developed and verified for several decades. Conversely, Kafe is able to exploit full-fledged kernel network stacks in a modern OS.

In addition, general networking frameworks such as netmap and Intel DPDK are actively maintained by many collaborators, but most user-level stacks do not show active maintenance [30]. In contrast, the network functions of a modern OS are regularly updated and actively maintained for performance, security, and additional functionalities. It is uncertain whether the user-level stacks can catch up with modern OSs in terms of up-to-date features. We expect Kafe to take advantage of active maintenance provided by modern OSs in the future.

Finally, the memory of user-space solutions is not protected [11], [16]. Application bugs are then able to corrupt the networking stack [16]. Furthermore, packets in the unprotected memory can be monitored or modified by an attacker [16], [31]. A recent study [31] applies Intel SGX [32] to DPDK to shield the unprotected memory region, but this approach adds considerable complexity to the networking stack. Kafe's memory is protected by the OS kernel, so that it is safer than user-space solutions.

III. Kafe Design Details

This section introduces the design goals, the main components including the skb Stack and the Flow Repository, and the packet forwarding path of Kafe.

A. Design Goals

The design goals of Kafe include performance, lightness, and scalability, which are explained as follows:

Performance: The primary goal of Kafe is to achieve high performance. For this purpose, Kafe inherits beneficial techniques from previous works of research. Fortunately, most of them are integrated into recent versions of Linux and their drivers. For example, the current *ixgbe* version supports multi-queue NICs to exploit parallelism [4].

In addition, Kafe tries to eliminate critical bottlenecks discussed in Section II-B. We analyze that these bottlenecks are mostly incurred by per-packet processing architectures in modern OSs. Modern OSs repeatedly perform buffer (de-)allocation, DMA address translation, packet inspection, routing lookup, and neighbor lookup during consecutive packet processing. Kafe tries to transform this per-packet processing design into a new recycle-based architecture that exploits pre-allocated and cached objects. This new design enables Kafe to process a packet without inefficient per-packet inspection and kernel object allocation. We show that Kafe can achieve performance comparable to the user-level approaches in Section IV-B.

Lightness: Current state-of-the-art approaches use an immense amount of resources to process incoming packets [7], [9], [13], [15], [33]. They adopt polling mechanisms and/or huge packet buffers at all times for boosting packet processing performance. However, this policy results in inefficient resource use, even when the rate of incoming packets is low. In Kafe, we avoid such a disadvantage by adopting the original NAPI architecture where the cores are utilized only when packets arrive at the system. In addition, Kafe tries to stay at low memory footprints by economizing memory use. We show that Kafe can provide lightweight packet processing in Section IV-C.

Scalability: We aim to offer scalable performance in terms of the number of cores and flows. For multi-core scalability, we adopt the multi-queue concept, in which each core has a pair of dedicated RX and TX queues. For dynamic load balancing between multi-queues, Kafe utilizes

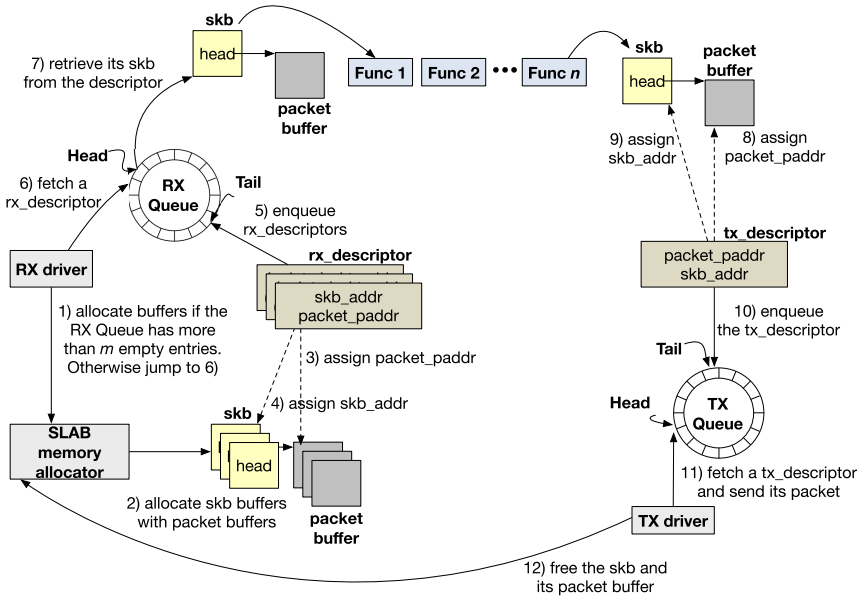

Fig. 2. skb and its packet buffer (de-)allocation procedure in Linux.

Receive Side Scaling (RSS), which distributes processing of received packets to multiple cores. As packets are distributed based on their flow information [20], [21], RSS preserves the order of received packets per flow. In addition, Kafe does not employ spinlocks in order to maximize parallel execution. Instead, it uses per-CPU variables to alleviate synchronization costs. For flow scalability, Kafe tries to support a sufficient number of flows to deal with increasing network traffic without performance degradation. We show that Kafe can satisfy scalability in terms of both the number of cores and flows in Section IV-D.

Reflecting these three design goals, Kafe develops the following mechanisms: the skb Stack and the Flow Repository.

B. skb Stack

As illustrated in Section II-B, in kernel packet processing, per-packet buffer allocation and DMA address translation lead to considerable performance deterioration. In this section, we investigate in depth how Linux (de-)allocates skb objects with their packet buffers and why this procedure incurs a significant amount of overhead. After this analysis, we develop a new cache-optimized buffer allocation mechanism called the skb Stack to boost skb and packet buffer allocation.

At the initialization time, Linux allocates a bundle of skb buffers along with their packet buffers in advance. An skb and its packet buffer are connected unidirectionally by assigning the address of the packet buffer to the *head* field of the skb. Allocated buffers are used to fill the RX queue. Each RX descriptor in the queue has a pointer to an skb (*skb_addr*) and a pointer to its packet buffer (*packet_paddr*). *packet_paddr* contains the DMA address of the packet buffer so that the NIC can copy the contents of an upcoming packet to this address.

Upon packet reception, the driver performs housekeeping and maintenance for the RX queue. As illustrated in Figure 2, the driver checks whether the RX queue has more than m empty entries (e.g., $m = 16$ in the default driver), which may have been de-queued by previous packet processing. If the queue has more than m empty entries, the driver allocates deficient skb and packet buffers through the SLAB memory allocator in order to replenish the RX queue. After this housekeeping takes place, the driver fetches a RX descriptor from the head of the queue for an incoming packet and reads the *skb_addr* field in the descriptor to access the allocated skb.

After the L3 layer functions are processed with this skb, the skb and its packet buffer are queued into the TX queue. The TX driver asynchronously fetches a TX descriptor at the head and sends the packet based on the DMA address of the packet buffer (*packet_paddr*). After the packet is confirmed to leave the host, the skb and its packet buffer are returned to SLAB. This procedure is repeated during consecutive packet processing. This per-packet scheme works satisfactorily when the input rate of packets is scarce. However, in the opposite case, it shows severe performance degradation because of the inefficient SLAB allocator and the DMA address translation cost as discussed in Section II-B.

We point out one more limitation that prevents effective packet processing. The problem is that the skb allocation procedure in modern OSs is not cache-optimized and can generate last-level cache (LLC) misses frequently. As shown in Figure 2, the SLAB allocator creates a new skb object for each incoming packet. The size of an skb is about 240 bytes and this will occupy four cache lines ($\lceil \frac{240\ bytes}{64\ bytes/line} \rceil$) in the last-level cache. During consecutive packet processing, the contents of each new skb must be brought into the cache, causing compulsory cache misses repeatedly. In addition, accessing every new skb each time will lead to capacity cache misses frequently, because an skb uses a non-trivial amount of memory. For example, after the driver processes 4,096 packets per core, the total amount of memory used for skb objects on

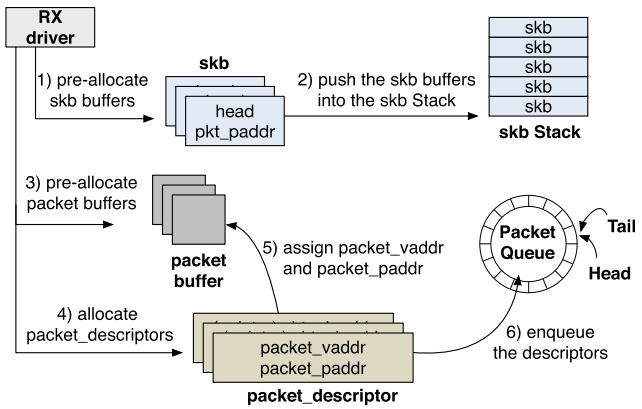

Fig. 3. Initialization process of the skb Stack.

all cores then reaches to approximately 8 MB on our eight core system, 40% the size of the L3 cache (20 MB). This large memory footprint will discard existing blocks from the L3 cache and thus generate capacity cache misses repeatedly.

To overcome these limitations, we develop a cache-optimized buffer allocation mechanism called the skb Stack. The main idea of the skb Stack is to store pre-allocated skb buffers in a stack data structure, instead of a queue, and recycle them continuously in packet processing. Because a stack operates on a last in, first out (LIFO) basis, recently pushed skb buffers are popped firstly from the stack while Kafe recycles skb buffers. As recently referenced buffers are likely to remain in the cache, using a stack can contribute to a decrease of LLC misses.

To realize the skb Stack concept, an skb and its packet buffer need to be connected after their packet is received. In Linux, the driver connects them at the allocation time (Figure 2) and immediately inserts them in the RX queue, which operates on a first in, first out (FIFO) basis. If the same mechanism is applied to Kafe, the FIFO behavior will weaken the cache effectiveness provided by the skb Stack. Therefore, the skb Stack does not connect the two buffers after allocation and only inserts the packet buffer in the RX queue. The skb Stack connects the two buffers after their packet reception. This is reasonable because an skb does not participate in the packet reception procedure performed by the NIC.

As shown in Figure 3, at the driver initialization time, the skb Stack pre-allocates a bundle of skb and packet buffers. It then saves the skb buffers in the skb Stack and the packet buffers in the Packet Queue respectively. The Packet Queue maintains packet descriptors to accommodate the virtual and DMA addresses of each packet. The virtual address field (*packet_vaddr*) is used to connect this packet to an skb at a later time. The virtual address is translated to the DMA address in advance to get rid of the translation cost during packet processing, and the latter is stored in the DMA address field (*packet_paddr*). In Figure 3, we add an additional field in the skb structure called *pkt_paddr* to store the DMA information after the two buffers are connected.

Figure 4 depicts how Kafe (de-)allocates skb objects with their packet buffers using the skb Stack. Upon packet reception, Kafe performs housekeeping for the RX queue. It checks whether the RX queue has empty entries as with the procedure illustrated in Figure 2. If the queue is not full, Kafe fetches several packet buffers from the Packet Queue in order to fill the RX queue. The information of each packet buffer including the virtual and DMA addresses is copied to each RX descriptor. Later, Kafe fetches a RX descriptor from the head of the RX queue for the incoming packet and retrieves its packet buffer. Simultaneously, Kafe pops an skb object from the skb Stack. Kafe connects the skb and the packet buffer by assigning *packet_vaddr* of the RX descriptor to *head* of the skb. Next, the skb is initialized with the contents of the RX descriptor and the packet header. Kafe also copies *packet_paddr* of the RX descriptor to *pkt_paddr* of the skb to boost DMA address translation during transmission. The relationship between the skb and its packet buffer is freed when the packet buffer is queued at the TX queue for transmission. The skb is then pushed into the skb Stack. The packet buffer is finally returned to the Packet Queue after its corresponding packet leaves the host.

The skb Stack alleviates heavy per-packet (de-)allocation and DMA address translation in modern OSs by recycling pre-allocated buffers. Owing to the use of a stack, the skb Stack is also cache-optimized. A piece of evidence is that Kafe utilizes a single (and the same) skb instance during consecutive packet forwarding in Figure 4. Modern OSs execute L3 forwarding functions in the *softirq* context, and for this purpose they launch a single *softirq* instance per core. The *softirq* instance then executes a successive forwarding function chain with the same skb parameter. In Kafe, when a packet arrives, the *softirq* instance pops an skb from the skb Stack and feeds the skb to the function chain. Before the packet leaves the host, the *softirq* instance pushes it into the skb Stack. When the next packet is processed, the *softirq* instance can pop the previous skb from the skb Stack. This pattern is repeated unless packets are delivered to other layers. Therefore, the skb Stack can produce exceptionally small memory footprints for cache efficiency.

The skb Stack is implemented on each core as per-CPU variables without locks to improve cache efficiency and eliminate synchronization costs. A shared memory data structure is generally managed by locks, but using locks leads to cache line bouncing [34] and decreases scalability due to lock contention between cores [35]. The per-CPU variable implementation overcomes these limitations by allowing each core to have its own copy of the skb Stack. In Kafe, each copy of the skb Stack manages its own skb buffers; pre-allocated skb buffers are not migrated to other cores by a load-balancing function. This design prevents stack overflows or underflows in each skb Stack. Although Kafe does not employ such a load-balancing function at the software level, incoming packets can be load-balanced across cores at the hardware level owing to RSS at NICs.

Compared to other buffer allocation mechanisms [4], [7], [8], [15], an advantage of the skb Stack is that it keeps the semantics of the skb data structure. Other mechanisms use their own and compact metadata structures, instead of the skb data structure, to minimize the initialization cost [7].

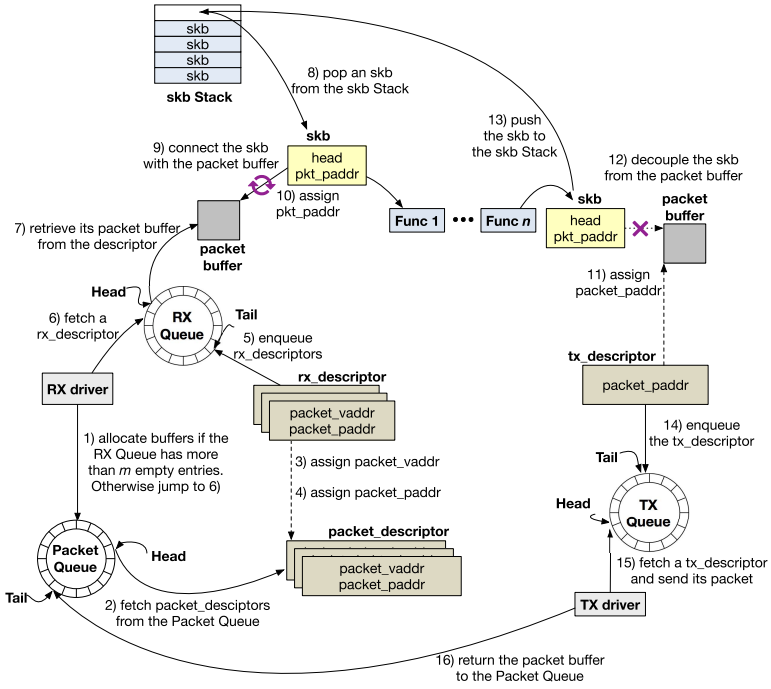

Fig. 4. skb and its packet buffer (de-)allocation procedure of the skb Stack in Kafe.

However, an skb is a fundamental data structure that most network functions in kernel space depend on. Therefore, forwarding applications with compact metadata cannot take advantage of full-fledged kernel network stacks when they require complex features such as firewall processing and IP fragmentation. As Kafe redesigns the network stack in kernel space, it keeps the semantics of the skb data structure.

C. Flow Repository

Section II-B reports that routing lookup, firewall processing, L3 layer protocol retrieving, and neighbor lookup can cause significant expenses. Each processing phase requires not only interpreting a packet, but also allocating its corresponding kernel objects from SLAB. This per-packet processing architecture causes significant performance deterioration similar to per-packet buffer allocation. We attempt to solve this issue by building a database called the Flow Repository. The Flow Repository stores kernel objects that can be recycled across multiple packets. Packets that belong to the same flow are entitled to reference to the cached kernel objects offered by the repository.

Linux and other modern OSs store this sort of information partially in separate cache subsystems. The examples are the routing cache subsystem before Linux 3.6 (which is deprecated because of performance issues) and the ARP cache. However, these cache subsystems are insufficient to cope with major bottlenecks in the network stack. The design of separate caches in modern OSs requires redundant cache entry lookup to find exact cache entries before using each cache system. The lookup operations have to occur several times per packet with separate cache subsystems. This will in turn consume non-negligible CPU resources [36]. Moreover, each cache data

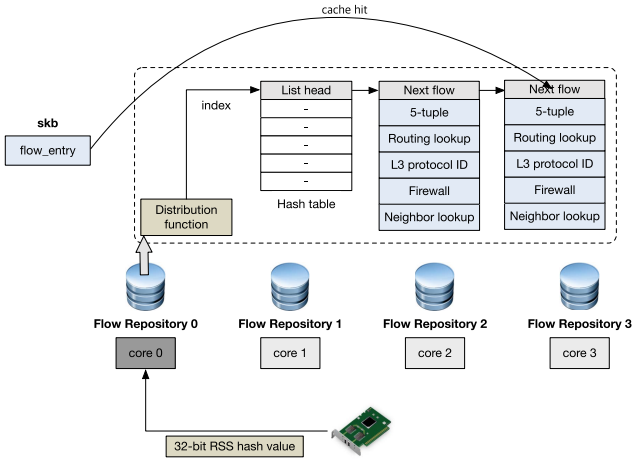

Fig. 5. Structure of the Flow Repository in Kafe.

structure demands synchronization methods to allow each core to access and modify the same entry exclusively, which severely deteriorates scalability.

The Flow Repository focuses on eliminating these two limitations. Figure 5 shows how the Flow Repository is implemented in Kafe. The Flow Repository maintains a hash table for flow classification. We use a hash table data structure because it can maintain performance on searching for cached objects even when the number of flows becomes quite large. A flow is assigned a hash index according to the 5-tuple values of the packet, which contains the source and destination addresses, the source and destination ports, and the L4 protocol identifier. A flow is placed on the hash table according to the hash index. As multiple different flows may have the same

hash index, we use a separate chaining technique with linked lists where each flow having the same hash index is connected by a list. A hash table entry that matches a flow provides cached objects to packets of the same flow. These include kernel objects regarding routing lookup, L3 layer protocol retrieving, firewall processing, and neighbor lookup. When the cache is hit, the Flow Repository can reduce the time taken for IP table lookup and other processing to $O(1)$.

To avoid the aforementioned redundant cache entry lookup operations in modern OSs, the Flow Repository implements a global cache system. The separate cache subsystems in modern OSs cause similar operations at every cache use. Concretely, each cache system redundantly executes a 5-tuple hash function and iterates a hash chain in order to find the target cache entry [36]. To avoid this redundancy, the Flow Repository implements a global cache system where the pointer of a hash table entry is given to its corresponding skb. For this purpose, we add an additional field, called *flow_entry*, in the skb structure. Then, each network function that deals with this skb can access the cached objects using the *flow_entry* field within a couple of memory references without spending time for looking up the cache entry.

To avoid the synchronization bottlenecks in modern OSs, the Flow Repository keeps per-CPU hash tables without locks. In Kafe, packets of the same flow are delivered to a specific core persistently by RSS, and this flow information is stored in the corresponding per-CPU hash table accessible by the target core. Because the hash table entry is only accessed by the dedicated core, there is no need to maintain synchronization methods to give other cores access to this hash entry. This design improves performance and scalability even when the number of cores in the system is increased.

The Flow Repository does not employ its own 5-tuple hash function to calculate the hash index from the 5-tuple values. Instead, it reuses the RSS hash value in the RX descriptor calculated by the NIC; modern NICs support this feature for facilitating RSS. The hash value provided by the RX descriptor is of 32 bits, but the size of the hash table in the Flow Repository is much more smaller than 2 to the power of 32. Therefore, we need a distribution function to perform a many-to-one mapping. The function simply divides the RSS value by the hash table size and takes the remainder as an index. This procedure is more lightweight than the way a 5-tuple hash function works and eliminates per-packet hash value calculation.

The Flow Repository removes each cache entry along with its cached kernel objects a few minutes after it is created. This helps the Flow Repository remain clean and updated as it periodically removes the information of short-lived flows and reflects changes in the routing table.

D. Packet Forwarding Using Kafe

To show the effectiveness of Kafe, this section explains a full packet forwarding path inside kernel space that adopts the skb Stack and the Flow Repository. To fully support IPv4 packet forwarding, we implement driver-level VLAN, IP header checksum validation, firewall processing, destination address interpretation, TTL decrement, and IP fragmentation.

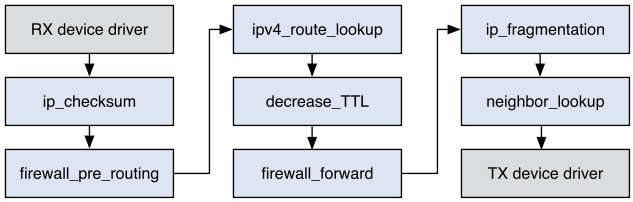

Fig. 6. Functional flow diagram of the packet forwarding path in Kafe.

As illustrated in Fig. 6, the packet forwarding path commences with the function call of the RX driver and finishes by calling the transmission function in the TX driver. *ip_checksum* (after the RX driver in Fig. 6) performs IP header checksum validation. *firewall_pre_routing* executes firewall processing before routing lookup. *ip_route_lookup* carries out destination address interpretation. In this function, packets bounded to the local host escape from the packet forwarding path and are transferred to the local delivery path (Section II-A). Currently, Kafe supports IPv4 routing lookup. *decrease_ttl* conducts TTL decrement. This function also re-computes the checksum value at the end because the TTL field resides in the header field. *firewall_forward* executes firewall processing after routing lookup. *ip_fragmentation* breaks a packet into smaller ones if the packet should enter a link with a smaller MTU. Finally, *neighbor_lookup* sets the MAC address field as the next hop connected by the output interface.

Each function in Fig. 6 utilizes Kafe when per-packet processing is required. The RX driver adopts the skb Stack to deal with buffer allocation for incoming packets. Other functions also refer to the Flow Repository when they process routing lookup, firewall processing, L3 layer protocol retrieving, and neighbor lookup. Each packet can pass through the forwarding path with just a few memory references when the repository has the cached objects. This cache hit also may be able to decrease the contention on the LLC because the network stack does not have to read the header of the packet buffer, which can decrease both compulsory and capacity cache misses on the LLC.

IV. Evaluation

In this section, we evaluate Kafe from the perspective of the design goals - performance, lightness, and scalability. After describing our test equipment and settings, this section presents the evaluation results.

A. Test Equipment and Settings

We deploy Kafe on commodity hardware with an Intel Xeon E5-2650 v2 2.6 GHz octa-core processor with hyper-threading off, 64 GB memory, a 256 GB SSD, and two dual-port Intel 82599EB 10GbE NICs. Because the cards have four ports, the aggregate throughput for RX and TX is 40 Gbps or 59.52 Mpps (million packets per second). We turn on eight RX and TX queues per NIC port, according to the number of CPU cores. The traffic generator is equipped with

two Intel Xeon E5-2630 v2 2.6 GHz hexa-core processors, 64 GB memory, a 3 TB HDD, and two dual-port Intel 82599EB 10GbE NICs. The Kafe server is directly connected to the traffic generator, which works as a packet source, as well as a sink, via four 10 GbE links. The packet generator software is based on the latest Pktgen (16.07) from WindRiver [25], which can generate network traffic up to 40 Gbps even with 64-byte packet size. We activate RSS on the both systems in order to fully exploit the multi-core architecture.

For comparison, we conduct the same packet forwarding evaluation on the same hardware using Linux, the Click kernel module [1], RouteBricks [4], Intel DPDK [15], and Kafe. Linux, Click, and RouteBricks are kernel-based whereas Intel DPDK adopts a user-level approach. Linux and the Intel *ixgbe* driver are with the version 3.16.4 and 3.22.3 respectively. The Click kernel module is based on the latest version (2.0.1). We use the plain Click configuration that enables simple L3 forwarding, which determines the egress interface of an incoming packet by checking the 16-bit prefix of the destination address. RouteBricks was ported to our equipment because current RouteBricks only supports old Intel Oplin 82598EB NICs. RouteBricks was configured to run on Linux 3.16.4, Click 2.0.1, and *ixgbe* 3.22.3. RouteBricks utilizes the Click kernel module as a forwarding engine; we apply the Click configuration that RouteBricks provides to the kernel module. Intel DPDK is the most powerful data plane compared to other user-level packet processing platforms because DPDK thoroughly exploits the Intel NIC hardware specification. In our evaluation, Intel DPDK accommodates the L3 forwarding sample application to forward packets using the 16-bit prefix.

Except for DPDK, we configure blank firewall rules for the firewall policy to forward packets without dropping them while applying the pre-routing, forward, and post-routing firewall functions to each packet. Even with blank firewall rules, the Linux kernel executes firewall-related functions (e.g., nf_iterate and ipt_do_table) per packet as shown in Table I.

For a realistic experiment condition, we populate the forwarding table with approximately 280k IP prefixes from the BGP table snapshot provided by RouteViews [37]. Except in the flow scalability evaluation presented in Section IV-D, we activate 8k flows in the traffic generator; the value is reasonable because the number of active flows in facilities such as enterprise and cloud data centers is normally less than 10k [38].

B. Performance

In this section, we evaluate the performance of Kafe and other solutions in terms of the throughut and latency.

Throughput: First, we measure the throughput of Kafe and other existing techniques while changing the generated packet size from 64 to 1514 bytes. The experimental result depicted in Fig. 7 shows that Kafe performs far better than Linux and the Click kernel module, for any packet size. Kafe can forward 64-byte IPv4 packets at 28.2 Gbps, which is seven times faster than Linux. In addition, Kafe can saturate 40 Gbps for packets larger than 256 bytes. The performance of Click is better than

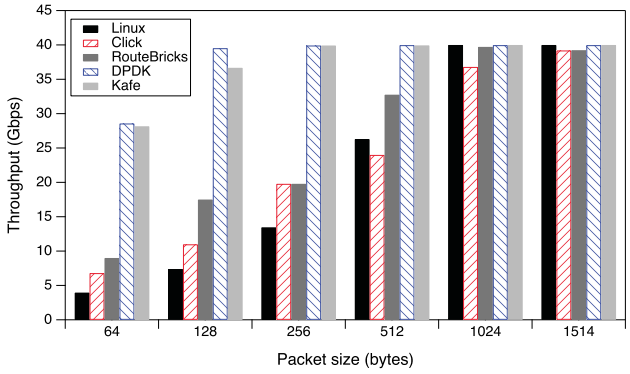

Fig. 7. Forwarding performance of Linux, Click, RouteBricks, DPDK, and Kafe with each packet size from 64 to 1514 bytes.

TABLE II
PERFORMANCE METRICS OF LINUX AND KAFE DURING PROCESSING 64-BYTE PACKETS

	Linux	Kafe with skb Stack	Kafe with skb Stack & Flow Repository
Throughput in Gbps (Mpps)	3.97 (5.90)	12.77 (19.00)	28.15 (41.89)
Cycles per packet	3,508.78	1,108.94	471.89
LLC misses per packet	4.39	0.55	0.77

that of Linux, as Click supports fast access to NICs through its components, without intervention of the kernel network stack. However, the forwarding performance is not sufficient (6 Gbps for 64-byte packets) to satisfy the line rate for packets smaller than 512 bytes. This is because Click still requires per packet memory allocation and inspection for destination address interpretation. RouteBricks achieves 9.0 Gbps with 64-byte packets, which is similar to the evaluation result in [4]. DPDK shows similar or slightly better performance compared to Kafe (28.6 Gbps for 64-byte packets, 39 Gbps or more for packets larger than 128 bytes). Considering that DPDK L3 forwarding does not perform essential functions such as IP header checksum validation, TTL decrement, and firewall processing described in Section III-D, we can claim that Kafe performs as fast as DPDK.

Second, we measure the performance impacts of the skb Stack and the Flow Repository in Kafe separately. Table II shows the throughput, CPU cycles per packet, and LLC misses per packet of Linux and Kafe during consecutive 64-byte packet processing. LLC misses per packet are obtained by dividing the total number of LLC misses, which was measured by the Linux Perf tools [39], by the total number of processed packets. Linux shows significantly high cycles and LLC misses per packet compared to Kafe. In particular, Linux generates more than four cache misses per packet, because the skb allocator in Linux is not cache-optimized as illustrated in Section III-B. To be specific, although an skb is usually hot in the LLC during packet processing [40], the skb allocator always returns a new skb object, which is not in the LLC. As an skb occupies four cache lines, at least four compulsory cache misses are unavoidable. On our machine, four cache misses take about 128 nanoseconds, and this is a considerable amount because a single packet needs to be forwarded within

TABLE III

AVERAGE LATENCY VALUE OF LINUX, DPDK, AND KAFE DURING CONSECUTIVE 64-BYTE PACKET PROCESSING

	Linux	DPDK	Kafe
Latency (micro-seconds)	3,120	352	574

about 67 nanoseconds to fully utilize the line speed with 64-byte packets. Kafe can improve the throughput (Gbps) by up to 221% only with the skb Stack compared to Linux. The skb Stack can remarkably decrease LLC misses per packet (i.e. from 4.39 to 0.55) as it produces small memory footprints by recycling skb objects. Combined with both the skb Stack and the Flow Repository, Kafe can improve the throughput (Gbps) by up to 608% compared to Linux; LLC misses per packet increase slightly compared to the skb Stack only case because the Flow Repository has additional memory accesses to the hash tables. The Flow Repository allows multiple packets from the same flow to recycle cached kernel objects for the flow. The Flow Repository reduces time taken for IP table lookup to $O(1)$ in most cases. In addition to this, as the Flow Repository utilizes cached objects in critical bottlenecks, it efficiently determines the destination port of packets without per-packet inspection and kernel object allocation.

Latency: Table III shows the average latency value of Linux, DPDK, and Kafe during consecutive 64-byte packet processing. Linux exhibits high latency because packet processing in the kernel layer incurs significant performance deterioration due to per-packet buffer allocation and per-packet routing lookup along with other additional processes as illustrated in Table I. Kafe dramatically improves this situation by employing the skb Stack and the Flow Repository, which allow each packet to pass through the L2 & L3 paths with just several memory references. DPDK shows outstanding latency because it adopts constant polling for incoming packets whereas Kafe employs NAPI, which combines interrupt and polling mechanisms. The use of interrupts in Kafe slightly increases latency, but it has an advantage, which will be discussed in Section IV-C.

C. Lightness

In this section, we evaluate the resource consumption in Kafe and other techniques with regard to the CPU cores and memory, respectively.

CPU Consumption: As depicted in Fig. 8, Linux requires a large amount of CPU cycles per packet, approximately 3k, regardless of the packet size. Since Linux shows limited forwarding rate of approximately 6 Mpps, irrespective of packet sizes, the number of CPU cycles per packet is shown to be constant. The number of CPU cycles needed for RouteBricks and DPDK increases exponentially with increase in the packet size. Because both RouteBricks and DPDK always adopt a polling mechanism for incoming packets, it requires 100% CPU utilization even when the rate of incoming packets is scarce, such as when the packet size exceeds 512 bytes. When compared to the other techniques, Kafe requires the least number of CPU cycles for all packet sizes. For 1024-byte packets, Kafe consumes only 1/4 of DPDK cycles. Because Kafe maintains the original NAPI mechanism, which uses CPU cores only when packets arrive at the system, Kafe does not waste CPU resources when there are few incoming packets.

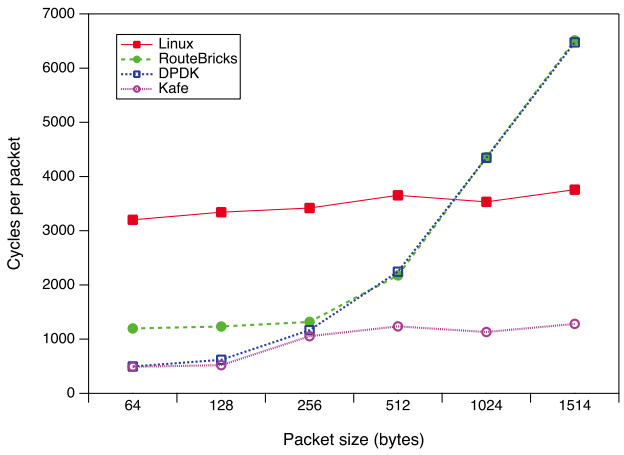

Fig. 8. Amount of CPU cycles consumed in forwarding a single packet with various packet sizes from 64 to 1514 bytes.

TABLE IV

AMOUNT OF MEMORY CONSUMED (MBYTES) IN FORWARDING PACKETS WITH VARIOUS PACKET SIZES FROM 64 TO 1514 BYTES

	64B	128B	256B	512B	1024B	1514B
Linux	393	426	431	436	437	437
RouteBricks	2,331	2,340	2,328	2,322	2,386	2,386
DPDK	33,955	33,957	33,957	33,956	33,956	33,956
Kafe	400	406	402	402	405	406

Memory Consumption: Table IV shows the total memory consumed in the same experiment depicted in Fig. 8. We find that the memory consumption of Kafe is similar to or less than that of Linux. Furthermore, the packet size does not affect the memory consumption of Kafe, which means that Kafe is able to maintain its lightness irrespective of the size of incoming packets. The memory consumption of RouteBricks is not affected by the packet size similar to Kafe. However, RouteBricks requires considerable memory because Click in RouteBricks continuously stores the pointers of incoming packets in the per-CPU array for packet batching. We see that the memory consumption of DPDK is extremely large (i.e. 33 GB), almost hundred times that of Kafe. This is because DPDK allocates huge pages in advance and stores incoming packets into the pages immediately. The minimum size of the required pages is 8 GB per 10 Gbps NIC, according to the performance report of Intel [15], and the value was configured in our experiments. Above this value, CPU cores seem to be saturated. Also, NetVM [11] reports that reduced huge page size brings performance degradation in DPDK. Compared to DPDK, Kafe significantly minimizes memory consumption owing to the skb Stack.

D. Scalability

The goals of Kafe include offering scalable performance in terms of the number of cores and flows.

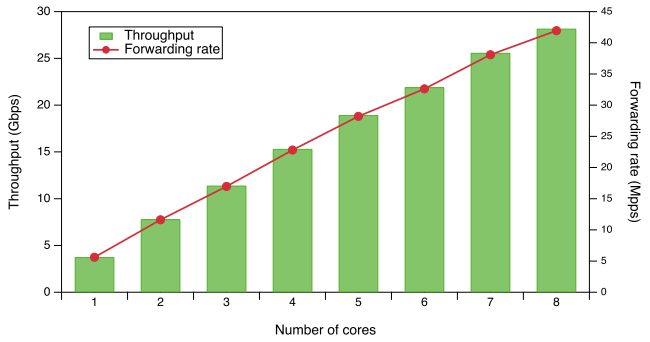

Fig. 9. Forwarding performance of Kafe with 64-byte packet size when the number of active cores increases from one to eight.

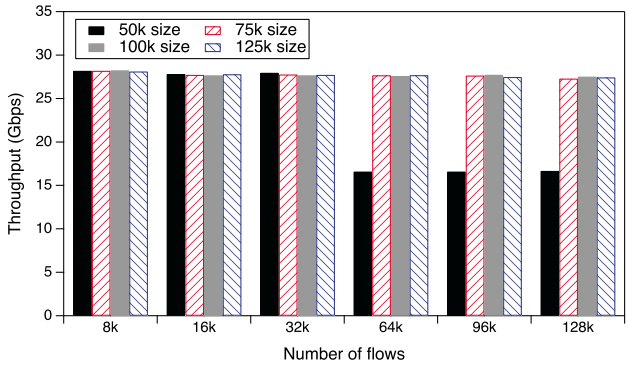

Fig. 10. Forwarding performance of Kafe when the size of the hash table is changed from 50k through 125k with each flow size.

Multi-Core Scalability: We evaluate the forwarding performance of Kafe by using different number of physical cores on the same machine. We change the number of active cores in the BIOS and conduct performance evaluation on packets of size 64 bytes. As depicted in Fig. 9, the forwarding performance of Kafe increases linearly with the number of physical cores. Kafe achieves 16.9 Mpps forwarding rate only with three physical cores, which exceeds the line rate (14.88 Mpps) for 64-byte packets in 10 Gigabit Ethernet. Since Kafe supports multi-queue NICs and does not employ spin locks in the forwarding path, each core can process incoming packets without intervention of other cores. In addition, because Kafe is not memory bound, the contention on LLC does not occur. This independence leads to linear performance increase when the number of physical cores increases.

Flow Scalability: As stated in Section III-A, Kafe aims to support a sufficient number of flows to deal with increasing network traffic. To evaluate this, we change the number of flows generated from the traffic generator from 8k to 128k with 64-byte packet size. The 128k value is quite large because it is ten times more than the number of active flows in enterprise and cloud data centers as reported in [38]. Flow scalability is closely related to the effectiveness of the Flow Repository. To validate Kafe's flow scalability, we measure the forwarding performance by varying the hash table size of the Flow Repository from 50k through 125k, and the results are depicted in Fig. 10. With the hash table size of 50k, when the number of packet flows exceeds 64k, the throughput of Kafe decreases from 28.2 Gbps to 16.5 Gbps, because hash collisions can occur frequently. However, as depicted in Fig. 10, when the hash table size is larger than or equal to 75k, Kafe can maintain high throughput, regardless of the number of flows. This result demonstrates that Kafe is scalable to a large number of packet flows with proper configuration of the Flow Repository.

V. Discussion

In this section, we discuss the portability, deployability, and resource consumption issues of Kafe in comparison with Intel DPDK in particular, a typical data plane for user-level solutions.

Portability: With regard to porting Kafe between different versions of the same OS, Kafe does not require significant effort. We have an experience of porting Kafe from Linux 3.16.4 to 4.4.21. The kernel data structures and functions required for Kafe remain pretty much the same in the later version. An exception is that functions starting with "ip_" in Linux 4.4.21 additionally receive the network layer representation of a socket (i.e., struct sock) as an argument. We dealt with this syntax revision without difficulty by processing the changed functions collectively. The lines of code that were modified for this porting is 50 out of 2,029 lines of Kafe's total code, which shows that the porting between different versions of the same OS is maintainable.

The porting of Kafe between different operating systems can be relatively easy provided that we define a compatibility layer that maps Linux-specific kernel functions to different OS equivalents. The Linux InfiniBand code was ported to FreeBSD using a similar layer without modifying the vast majority of the driver code [41]. Intel adopts a similar approach for better portability of DPDK. DPDK defines the Environment Abstraction Layer (EAL) [42], which provides access to low-level resources such as CPU cores, memory space, PCI devices, timers, and consoles. However, as the size of the EAL code is quite huge (i.e., approximately 25,000 lines), it would be difficult to port DPDK to other operating systems. In comparison with DPDK, Kafe has much less code, so that the porting would be much easier than DPDK.

Deployability: For deployment, Kafe currently requires recompilation of the kernel source code and a system reboot, as with other kernel-based approaches such as RouteBricks [4]. For alleviation of its deployment burden, Kafe can be also implemented as a loadable kernel module (LKM), which does not require a reboot, and the module can be distributed as a binary file to avoid recompilation.

DPDK adopts a user-level approach, but its deployment burden is not lightweight. According to DPDK's Getting Started Guide for Linux [43], a user is required to reboot the system in the beginning to prevent the allocated huge pages from being fragmented. Next, the user needs to configure huge pages, compile the DPDK source code, and install the compiled libraries, drivers, and applications. In addition, the Linux kernel needs to be recompiled in several Linux distributions to use the High Precision Event Timer (HPET) in user applications. These steps require more time and careful attention than Kafe.

TABLE V
SUMMARY OF THE CHARACTERISTICS OF SEVERAL PACKET PROCESSING FRAMEWORKS

	RouteBricks (2009)	PacketShader (2010)	Netmap (2012)	DPDK (2013)	IX (2014)	Kafe
Privilege level	Kernel	User	User	User	Kernel + User	Kernel
Buffer allocation	* No use of skb * Buffer reuse	* No use of skb * Huge packet buffers	* No use of skb * Fixed size packet buffers	* No use of skb * Exploiting huge pages	* Replacing skb to pre-allocated mbufs	* Use of skb * ***skb Stack***: cache-optimized allocator
Flow classification	X	X	X	* Hash table	* Utilizing RSS flow mapping	**** Flow Repository***: scalable flow cache

Resource Consumption: Performance is the most essential factor in software routers, but memory and CPU consumption are also important when a software router is deployed on a commodity PC or in a virtualized environment. Researchers have traditionally focused on deployment of software routers on commodity PCs for cost-effectiveness [7], [44]. However, recent commodity PCs still have 4–8 GB of main memory. This means that PCs with DPDK would not provide competitive performance due to lack of memory. In contrast, Kafe can keep its high performance in a commodity PC because it only spends approximately 400 MB of memory for packet forwarding.

Recent network virtualization and network functions virtualization (NFV) have catalyzed deployment of software routers on virtual machines (VMs) in cloud computing [45]. The problem with high memory consumption is that cloud providers charge the cost according to how much memory is provided on each VM. Therefore, a VM deploying DPDK can cause high expenses in cloud computing. High CPU consumption also matters in a virtualized environment. DPDK's continuous polling in a VM wastes its allocated CPU time given by the hypervisor's CPU scheduler, even if there is no packet to process. The VM then suffers from insufficient CPU time for processing other incoming packets. This highly CPU-intensive characteristic can decline the performance of a VM with DPDK in terms of throughput and latency [46]. Conversely, Kafe reserves its allocated CPU for future packets when there is no incoming packet. This behavior does not hamper the performance in a virtualized environment.

VI. RELATED WORK

User-Level Approach: Some approaches [7]–[9], [13]–[15], [33] accelerate packet processing by directly accessing the NICs in user space.

PacketShader [7] additionally exploits the processing capacity of GPUs to address the CPU bottlenecks in software routers. They also put efforts into solving the problem of the multi-core management. For instance, they resolve the false sharing problem by aligning every starting address of per-queue data to the cache line boundary. With the support of GPUs, the forwarding performance reaches 39 Gbps for IPv4 with 64-byte packets in two dual port 10 GbE NICs. Without GPU support, this router can achieve approximately 28 Gbps using 8 cores running at 2.66 GHz.

Kim *et al.* [9] introduced an efficient batching technique and multi-queue NIC support to the user-level Click router. The totally modified Click router achieves 28 Gbps for 64-byte packets on the eight core machine running at 2.66 GHz.

netmap [8] demonstrated considerable performance improvement when receiving or sending packets by relieving per-packet dynamic memory allocation, system call overheads, and memory copy. As netmap is a packet processing engine, it provides a user-space library named *libpcap*. With the help of this library, the user-space Click application can achieve large speed-ups, up to five times that of the original.

Intel DPDK [15] provides high throughput and low latency packet processing by eliminating overheads inherent in interrupt driven OS-level packet processing. Although DPDK shows outstanding forwarding performance, its resource consumption is immense because of the steady polling mechanism and the use of huge pages, as illustrated in Fig. 8 and Table IV. The massive resource consumption makes it difficult to adopt this technology to small and medium enterprises that consider the management cost an important factor.

These user-level approaches provide remarkable performance by bypassing the bottlenecks intrinsic to modern OSs. However, they are vulnerable to activities occurring in the user processes. Moreover, they can potentially face security problems because the data plane memory is exposed to user space while security features are almost absent. On the contrary, Kafe can provide a reliable environment owing to the conventional kernel-based networking stack.

Kernel-Level Approach: Some research focused on the enhancement of packet processing in the kernel level.

Bianco *et al.* [2] claim that the main bottlenecks of Linux in packet processing are - receive livelocks and excessive delay in skb allocation. Receive livelocks stem from a race condition between the hardware and software interrupt handlers [47]. In Kafe, the receive livelock issue can be avoided by adopting NAPI, and the skb allocation problem can be resolved by the skb Stack.

Bolla and Bruschi [3] presented enhanced network performance of Linux on multi-processor architectures. To fully exploit this architecture, they used advanced hardware, an Intel Advanced Smart Cache and Intel PRO 1000 network adapter, to reduce cache misses and to bind processors to the RX and TX rings of each network interface.

RouteBricks [4] parallelized router functionalities within multi-cores and across multiple servers. As with [3], each

queue of a network interface was bound to a different core for better CPU utilization. Additionally, they chose NIC-driven batching to receive/send packets from/to NICs in batches. The forwarding performance measured on a single server was 8.3 Gbps with routing table lookup and 12.7 Gbps without it, given 64-byte packets on the 2.8 GHz eight core system.

IX [16] provides a library OS specialized in processing the TCP/IP protocol. It realizes strong protection by accommodating the networking stack in the kernel layer as with Kafe. IX asserts that the existing user-level approaches have a tradeoff between performance and security and indicates that an attacker can corrupt the networking functions and their packet buffers. In terms of implementation, IX develops proprietary network functions that are not compatible with existing Linux. However, Kafe can coexist with Linux by preserving the skb interface in every network function.

Table V briefly summarizes the characteristics of several previous works, and compares them with Kafe. Compared to the previous research, Kafe maximizes kernel-level packet processing performance and minimizes resource consumption by applying the new schemes: the skb Stack and the Flow Repository.

VII. Conclusions

This paper attempts to contradict a belief that kernel network stacks are inefficient in software routers. Through proper implementation, Kafe is able to achieve packet forwarding performance as fast as user-space network stacks. The two key schemes in Kafe, the skb Stack and the Flow Repository, eliminate major bottlenecks of packet processing in modern OSs. Owing to these techniques, Kafe can outperforms Linux by seven times and RouteBricks by three times. In addition, Kafe can achieve other critical goals including lightness and scalability thanks to its thin and robust architecture.

The current version of Kafe fully supports IPv4 packet forwarding. However, IPv6 packet processing can also take advantage of Kafe without difficulty, because Kafe optimizes kernel network stacks while preserving existing data structures. Current Kafe provides high speed packet processing for the L2 and L3 layers. In addition, we are working on optimizing Kafe with the L4 layer protocols including TCP and UDP in order to enlarge the advantages of Kafe. We hope that our research will contribute toward further studies on software routers based on commodity hardware and modern OSs.

Acknowledgment

The authors are grateful to the anonymous reviewers for their valuable comments and suggestions.

References

[1] E. Kohler, R. Morris, B. Chen, J. Jannotti, and M. F. Kaashoek, "The click modular router," *ACM Trans. Comp. Syst.*, vol. 18, no. 3, pp. 263–297, Aug. 2000.

[2] A. Bianco *et al.*, "Click vs. Linux: Two efficient open-source IP network stacks for software routers," in *Proc. Workshop High Perform. Switching Routing (HPSR)*, May 2005, pp. 18–23.

[3] R. Bolla and R. Bruschi, "PC-based software routers: High performance and application service support," in *Proc. ACM Workshop Program. Routers Extensible Services Tomorrow*, 2008, pp. 27–32.

[4] M. Dobrescu *et al.*, "RouteBricks: Exploiting parallelism to scale software routers," in *Proc. ACM SIGOPS 22nd Symp. Operat. Syst. Princ.*, 2009, pp. 15–28.

[5] Y. Ma, S. Banerjee, S. Lu, and C. Estan, "Leveraging parallelism for multi-dimensional packetclassification on software routers," *ACM SIGMETRICS Perform. Eval. Rev.*, vol. 38, no. 1, pp. 227–238, 2010.

[6] J. Zhao, X. Zhang, X. Wang, Y. Deng, and X. Fu, "Exploiting graphics processors for high-performance IP lookup in software routers," in *Proc. IEEE INFOCOM*, 2011, pp. 301–305.

[7] S. Han, K. Jang, K. Park, and S. Moon, "PacketShader: A GPU-accelerated software router," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 41, no. 4, pp. 195–206, 2011.

[8] L. Rizzo, "Netmap: A novel framework for fast packet I/O," in *Proc. USENIX Annu. Tech. Conf.*, 2012, pp. 101–112.

[9] J. Kim, S. Huh, K. Jang, K. Park, and S. Moon, "The power of batching in the click modular router," in *Proc. Asia–Pacific Workshop Syst.*, 2012, pp. 1–14.

[10] J. Martins *et al.*, "ClickOS and the art of network function virtualization," in *Proc. 11th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2014, pp. 459–473.

[11] J. Hwang, K. Ramakrishnan, and T. Wood, "NetVM: High performance and flexible networking using virtualization on commodity platforms," in *Proc. 11th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2014, pp. 1–15.

[12] T. Brecht, G. J. Janakiraman, B. Lynn, V. Saletore, and Y. Turner, "Evaluating network processing efficiency with processor partitioning and asynchronous I/O," *ACM SIGOPS Oper. Syst. Rev.*, vol. 40, no. 4, pp. 265–278, 2006.

[13] N. Bonelli, A. Di Pietro, S. Giordano, and G. Procissi, "PFQ: A novel architecture for packet capturing on parallel commodity hardware," in *Proc. 9th Italian Netw. Workshop*, 2012. [Online]. Available: https://www.telematica.polito.it/oldsite/Italian_Networking_Workshop_2012/index.html

[14] ntop. (2011). *Pf_ring*. [Online]. Available: http://www.ntop.org/products/packet-capture/pf_ring/

[15] Intel. *Data Plane Development Kit*. Accessed: Dec. 15, 2016. [Online]. Available: http://dpdk.org

[16] A. Belay *et al.*, "IX: A protected dataplane operating system for high throughput and low latency," in *Proc. 11th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2014, pp. 49–65.

[17] C. Rotsos, N. Sarrar, S. Uhlig, R. Sherwood, and A. W. Moore, "OFLOPS: An open framework for OpenFlow switch evaluation," in *Passive and Active Measurement*. Vienna, Austria: Springer, 2012, pp. 85–95. [Online]. Available: https://www.springer.com/la/book/9783642285363

[18] C. Benvenuti, *Understanding Linux Network Internals*. Newton, MA, USA: O'Reilly Media, 2006.

[19] *Intel 82599 10 Gigabit Ethernet Controller: Datasheet, Revision: 2.73*, Intel, Santa Clara, CA, USA, 2011.

[20] *Receive Side Scaling on Intel Network Adapters*, Intel, Santa Clara, CA, USA, 2016.

[21] P. Shinde, A. Kaufmann, T. Roscoe, and S. Kaestle, "We need to talk about NICs," in *Proc. 14th USENIX Conf. Hot Topics Oper. Syst.*, 2013, pp. 1–8.

[22] D. Freimuth *et al.*, "Server network scalability and TCP offload," in *Proc. USENIX Annu. Tech. Conf., General Track*, 2005, pp. 209–222.

[23] B. H. Leitao, "Tuning 10 Gb network cards on Linux," in *Proc. Linux Symp.*, 2009, pp. 169–184.

[24] F. Baker, *Requirements for IP Version 4 Routers*, document RFC 1812, 1995. [Online]. Available: https://tools.ietf.org/html/rfc1812

[25] WindRiver. *The Pktgen Application*. [Online]. Available: http://pktgen.readthedocs.org/en/latest/index.html

[26] T. B. Ferreira, R. Matias, A. Macedo, and B. Evangelista, "An experimental comparison analysis of kernel-level memory allocators," in *Proc. 30th Annu. ACM Symp. Appl. Comput.*, 2015, pp. 2054–2059.

[27] D. Siemon, "Queueing in the Linux network stack," *Linux J.*, vol. 2013, no. 231, p. 2, 2013.

[28] E. Jeong *et al.*, "mTCP: A highly scalable user-level TCP stack for multicore systems," in *Proc. NSDI*, 2014, pp. 489–502.

[29] I. Marinos, R. N. Watson, and M. Handley, "Network stack specialization for performance," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 175–186, 2014.

[30] K. Yasukata, M. Honda, D. Santry, and L. Eggert, "StackMap: Low-latency networking with the OS stack and dedicated NICs," in *Proc. USENIX Annu. Tech. Conf.*, 2016, pp. 43–56.

[31] B. Trach, A. Krohmer, F. Gregor, S. Arnautov, P. Bhatotia, and C. Fetzer, "ShieldBox: Secure middleboxes using shielded execution," in *Proc. Symp. SDN Res.*, 2018, p. 2.
[32] V. Costan and S. Devadas, "Intel SGX explained," IACR Cryptol. ePrint Arch., Tech. Rep. 2016/86, 2016.
[33] N. Bonelli, A. Di Pietro, S. Giordano, and G. Procissi, "On multi–gigabit packet capturing with multi–core commodity hardware," in *Passive and Active Network Measurement*. Vienna, Austria: Springer, 2012, pp. 64–73.
[34] R. Olsson, "Pktgen the Linux packet generator," in *Proc. Linux Symp.*, Ottawa, ON, Canada, vol. 2, 2005, pp. 11–24.
[35] S. Boyd-Wickizer *et al.*, "An analysis of Linux scalability to many cores," in *Proc. OSDI*, vol. 10, no. 13, 2010, pp. 86–93.
[36] H. Song, S. Dharmapurikar, J. Turner, and J. Lockwood, "Fast hash table lookup using extended Bloom filter: An aid to network processing," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 4, pp. 181–192, 2005.
[37] A. Broido and K. C. Claffy, "Analysis of RouteViews BGP data: policy atoms," in *Proc. Netw.-Rel. Data Manage. Workshop*, Santa Barbara, CA, USA, May 2001. [Online]. Available: http://www.caida.org/publications/papers/2001/NdrmBgp/
[38] T. Benson, A. Akella, and D. A. Maltz, "Network traffic characteristics of data centers in the wild," in *Proc. 10th ACM SIGCOMM Conf. Internet Meas.*, 2010, pp. 267–280.
[39] A. C. De Melo, "The new Linux 'perf' tools," in *Proc. Slides Linux Kongress*, vol. 18, 2010, pp. 1–42.
[40] J. D. Brouer, "Network stack challenges at increasing speeds," in *Proc. Linux Conf.*, Auckland, New Zealand, 2015, pp. 12–16.
[41] J. Roberson. *Linux Kernel Compatibility*. Accessed: Jan. 15, 2018. [Online]. Available: https://lwn.net/Articles/421605/
[42] Intel. *Environment Abstraction Layer for DPDK*. Accessed: Jan. 15, 2018. [Online]. Available: http://dpdk.org/doc/guides/prog_guide/env_abstraction_layer.html
[43] Intel. *Intel DPDK Getting Started Guide for Linux*. Accessed: Jan. 15, 2018. [Online]. Available: http://dpdk.org/doc/guides-17.11/linux_gsg/index.html
[44] A. Bianco, J. M. Finochietto, G. Galante, M. Mellia, and F. Neri, "Open-source PC-based software routers: A viable approach to high-performance packet switching," in *Quality of Service in Multiservice IP Networks*. Catania, Italy: Springer, 2005, pp. 353–366. [Online]. Available: https://www.springer.com/us/book/9783540245575
[45] R. Jain and S. Paul, "Network virtualization and software defined networking for cloud computing: A survey," *IEEE Commun. Mag.*, vol. 51, no. 11, pp. 24–31, Nov. 2013.
[46] H. Kim, H. Lim, J. Jeong, H. Jo, and J. Lee, "Task-aware virtual machine scheduling for I/O performance," in *Proc. ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environ.*, 2009, pp. 101–110.
[47] J. C. Mogul and K. K. Ramakrishnan, "Eliminating receive livelock in an interrupt-driven kernel," *ACM Trans. Comput. Syst.*, vol. 15, no. 3, pp. 217–252, 1997.

Cheol-Ho Hong received the B.S., M.S., and Ph.D. degrees in computer science from Korea University, Seoul, South Korea. He was a Research Professor with Korea University from 2013 to 2015 and a Research Fellow with Queen's University Belfast from 2015 to 2018. He is currently an Assistant Professor with the School of Electrical and Electronics Engineering, Chung-Ang University, Seoul. His current research interests include operating systems, system virtualization, data center networks, software-defined networking, and GPU architectures.

Kyungwoon Lee received the B.E. degree from the School of Electronic engineering from Kyungpook National University, South Korea, in 2010, and the M.S. degree in computer science from Korea University, Seoul, South Korea, in 2015, where she is currently pursuing the Ph.D degree with the Operating Systems Laboratory. Her research interests include network scheduling in cloud computing, software-defined networking, and system virtualization.

Jaehyun Hwang received the B.S. degree in computer science from The Catholic University of Korea, South Korea, in 2003, and the M.S. and Ph.D. degrees in computer science from Korea University, Seoul, South Korea, in 2005 and 2010, respectively. He was with Bell Labs, Alcatel-Lucent, Seoul, as a Member of Technical Staff from 2010 to 2015. He has been with Samsung Electronics, South Korea, as a Senior Engineer since 2015. His current research interests include data center networks, software-defined networking, multipath TCP, and HTTP adaptive video streaming.

Hyunchan Park received the B.S., M.S., and Ph.D. degrees in computer science from Korea University, South Seoul. He was a Research Professor with Korea University from 2014 to 2016. He is currently an Assistant Professor with the Division of Computer Science and Engineering, Chonbuk National University. His current research interests include storage systems especially with flash-based devices, virtualization systems, and operating systems.

Chuck Yoo (M') received B.S. and M.S. degrees in electronic engineering from Seoul National University, Seoul, South Korea, and the M.S. and Ph.D. degrees in computer science from the University of Michigan. He was a Researcher with the Sun Microsystems Laboratory from 1990 to 1995. He has been a Professor with the College of Information, Korea University, Seoul, South Korea, since 1995. His research interests include operating systems, embedded system, virtualization, and network virtualization. He is a member of the IEEE Computer Society and the ACM.